Proactive Reinforcement Learning for Robust Quadruped Locomotion

Under Limb Dropout and Sensor Noise

School of Computer Science & Applied Mathematics
University of the Witwatersrand

Anand Patel
Student Number: 2561034

Supervised by:
Benjamin Rosman
Steven James

May 26, 2025

A proposal submitted to the Faculty of Science, University of the Witwatersrand, Johannesburg,
in partial fulfilment of the requirements for the degree of Bachelor of Science with Honours

Abstract

Real-world deployment of quadruped robots demands high robustness to internal hardware failures and sensor degradation—conditions that are often overlooked during conventional reinforcement learning (RL) training. This project investigates the use of proactive robustness strategies to train locomotion policies capable of maintaining forward motion in the presence of joint failures and sensor noise. Using the RealAnt quadruped robot as the experimental platform, we propose a training pipeline that integrates curriculum-based domain randomization with Smooth Regularized Reinforcement Learning (SR^2L) on top of a Proximal Policy Optimization (PPO) backbone.

The robot is trained under progressively more challenging failure scenarios, beginning with simple behavior and advancing to randomized actuator dropout and noisy sensor readings. SR^2L further improves stability by encouraging smooth policy responses to perturbed observations, mitigating the risk of unpredictable behavior due to degraded joint and motion feedback.

Performance is evaluated against ablated variants of the proposed method—including policies trained with only domain randomization, only smooth regularization, only proximal policy optimization (PPO) or neither—to isolate each component's contribution to robustness. This allows a detailed analysis of each component's contribution to robustness. Evaluation is conducted in simulation using diverse fault scenarios, with metrics including success rate, recovery time, motion stability, and failure frequency. By combining principled regularization and fault-aware training, this research contributes a robust control strategy applicable to resource-constrained platforms operating in mission-critical or hazardous environments. The work also lays a foundation for future exploration in terrain-aware planning, vision-based adaptation, and sim-to-real transfer.

Declaration

I, Anand Patel, declare that this proposal is my own, unaided work. It is being submitted for the degree of **BSc (Hons) Computer Science** at the University of the Witwatersrand, Johannesburg. It has not been submitted for any degree or examination at any other university.

Anand Patel
May 26, 2025

Acknowledgements

I would like to thank my supervisors Dr. Benjamin Rosman, and Dr. Steven James, for their guidance and continuous support throughout this project.

Anand Patel
May 26, 2025

Contents

Preface 0

Abstract 1

Declaration 2

Acknowledgements 3

1 Introduction 6

2 Background and Related Work 7

2.1 Reinforcement Learning for Robotic Locomotion 7
2.2 Domain Randomization 8
2.3 Smooth Regularized Reinforcement Learning 9
2.4 Robust Locomotion under Hardware Faults 9
2.5 Research Question 10
2.6 Hypothesis 10
2.7 Research Gap 11

3 Methodology 13

3.1 Simulation Setup 13
3.2 Policy Architecture and Algorithms 14
3.3 Fault Injection: Limb Dropout and Sensor Noise 15
3.4 Training Procedures 17
3.5 Ablation Baselines 20
3.6 Stretch Goals and Extended Objectives 21

4 Evaluation 22

4.1 Evaluation Metrics 22
4.2 Experimental Setup 23
4.3 Result Analysis Strategy 23
4.4 Interpretation and Limitations 24

4.5 Timeline and Milestones 24

5 Limitations **26**

6 Conclusion **27**

Chapter 1

Introduction

Quadruped robots are increasingly being used in environments where mobility and robustness are crucial — such as search-and-rescue operations, industrial inspections, and space exploration. These applications require not only precise locomotion but also the ability to remain functional under hardware degradation and sensor unreliability. Despite advances in reinforcement learning (RL), most locomotion policies are trained under ideal conditions and lack the robustness required for real-world deployment (Hwangbo et al., 2019). In particular, small robots with limited onboard computation, such as the Ant platform, are vulnerable to actuator failures (e.g., joint locking) and noisy sensors, both of which can destabilize the behavior of the robot. To address this, recent research has explored proactive methods such as domain randomization, where faults are introduced during training to encourage robustness (Tobin et al., 2017; Peng et al., 2020). Additionally, smooth regularization techniques such as SR^{2}L have shown great results in improving policy stability under perturbations (Liu et al., 2023b). This proposal investigates whether combining these proactive strategies — specifically Domain Randomization, SR^{2}L and PPO — can produce a locomotion policy that outperforms standard RL baselines in terms of robustness to limb dropout and sensor noise. The resulting findings aim to contribute to the development of fault-tolerant learning systems suitable for long-term autonomy in unstructured environments.

Chapter 2

Background and Related Work

2.1 Reinforcement Learning for Robotic Locomotion

Reinforcement Learning (RL) is a subfield of machine learning where an agent learns to make decisions by interacting with an environment to maximize cumulative reward. Formally, RL problems are modeled as Markov Decision Processes (MDPs), consisting of a state space, action space, transition dynamics, and a reward function (Sutton and Barto, 2018). The agent observes the current state then selects an action based on a policy, and receives a scalar reward along with the next state. Over time, it updates its policy to maximize the expected return.

Policies are typically represented as parameterized functions (often neural networks in deep RL) that map observations to actions. Value functions estimate the expected reward of being in a certain state or taking a specific action, and are used in actor-critic methods like PPO to guide policy updates. These learning mechanisms make RL especially suitable for complex control problems where accurate system modeling is difficult or impractical. Reinforcement learning (RL) provides a powerful framework for training autonomous agents to learn complex behaviors through trial-and-error interactions with their environment. In robotic control, RL allows policies to be optimized from observation to continuous actions, without requiring manually control rules (Sutton and Barto, 2018). This approach is useful for locomotion tasks, where modeling the full dynamics of a robot and its interaction with the environment can be intractable and sometimes unreproducible

Legged robots are challenging to control due to their limited actuation relative to their degrees of freedom. Traditional model-based control often fails to scale to these systems, while RL has shown it's ability to learn robust locomotion strategies directly from physics simulations (Hwangbo et al., 2019; Haarnoja et al., 2018). In particular, actor-critic methods such as Proximal Policy Optimization (PPO) (Schulman et al., 2017) have

become widely used due to their stability and sample efficiency in high-dimensional control settings. PPO (Proximal Policy Optimization) updates the policy in small, stable steps by maximizing a carefully clipped objective function. This clipping limits how much the new policy is allowed to deviate from the previous one during each update, which helps prevent instability and catastrophic performance drops during training.

Recent applications have shown that RL can be used to train locomotion policies for quadrupeds that generalize to a variety of walking, running, and agile movement behaviors (Hwangbo et al., 2019; Lee et al., 2020; Miki et al., 2022). However, most of these policies are sensitive to differences between training and deployment conditions, especially when sensor noise or hardware faults are present — presenting the need for more robust training techniques, explored in subsequent sections.

2.2 Domain Randomization

Domain Randomization (DR) is a technique used to improve the transferability and robustness of reinforcement learning policies by introducing structured variety during training. Rather than training a policy in a single fixed environment, DR exposes the agent to a wide range of simulated conditions, such as changes in friction, mass, sensor noise, and actuator characteristics (Tobin et al., 2017). The goal is to prevent overfitting to any specific configuration, and in turn encouraging the development of policies that generalize across a wider range of environments and internal states and scenarios.

In robotics, DR has shown to be useful for sim-to-real transfer, where policies trained in simulation are deployed on physical hardware. By randomizing parameters such as terrain properties, visual textures, or dynamics, researchers have successfully trained policies that are robust to differences between simulation and the real world (Peng et al., 2018; Sadeghi and Levine, 2016). Recent work has extended this idea to fault tolerance, where damage scenarios, such as actuator failures, are injected during training to produce policies that are resilient to physical degradation (Liu et al., 2023a).

For locomotion tasks, DR has been applied to quadruped and humanoid robots to enable recovery from unexpected disturbances, improve locomotion stability, and maintain forward motion even when limbs are degraded (Margolis et al., 2022). The effectiveness of DR comes from its simplicity and its compatibility with widely-used policy optimization algorithms such as Proximal Policy Optimization (PPO), without requiring architectural modifications. However, it can sometimes lead to overly conservative policies if the range of randomized conditions is too broad bringing to light the need for correct design of randomization parameters.

2.3 Smooth Regularized Reinforcement Learning

While Domain Randomization improves policy robustness by exposing agents to a variety of training scenarios, it does not explicitly address the sensitivity of policies to small, continuous perturbations in input. In real-world robotics, sensor noise and low-frequency feedback disruptions are common, and can lead to abrupt or unstable behaviors in trained policies. To address this issue, Smooth Regularized Reinforcement Learning (SR^2L) introduces a regularization term to the policy loss that explicitly penalizes large changes in the action output when the state input is perturbed slightly (Liu et al., 2023c).

SR^2L builds on the understanding that robust policies should produce consistent and smooth outputs under minor variations in observations. SR^2L introduces an additional smoothness term to the standard policy gradient objective, encouraging the policy to produce similar actions for similar input states. This constraint, known as Lipschitz continuity, ensures that small changes in sensor readings do not cause large, abrupt changes in the robot's behavior. As a result, the policy becomes more stable and less sensitive to noise or delay in sensory feedback.

In combination with PPO, SR^2L has been shown to improve training stability, generalization, and fault tolerance in dynamic control settings (Liu et al., 2023c). Although originally proposed for settings with noisy, uncertain dynamics and sensors, its use in locomotion control is motivated by the need for smooth torque profiles and stable movement patterns, especially under partial observability or sensor degradation. In this study, SR^2L is integrated with Domain Randomization to improve both high-level robustness to failure scenarios and low-level stability in the presence of observation noise.

2.4 Robust Locomotion under Hardware Faults

Hardware faults, such as actuator failure or sensor degradation, present challenges to the deployment of autonomous legged robots in real-world environments. These faults may arise due to wear-and-tear, environmental damage, or design limitations — and often result in partial or total loss of control in one or more joints, as well as inaccurate or noisy observations from onboard sensors. Without explicit robustness training, policies optimized under ideal conditions are highly sensitive to such disruptions, often failing catastrophically when deployed outside their training environment (Haarnoja et al., 2018; Rajeswaran et al., 2017).

In quadruped locomotion, actuator failure is typically modeled as joint dropout, where one or more motors are locked in place or disabled. Several studies have demonstrated

that introducing such faults during training can lead to the development of compensatory strategies (Jain et al., 2020). Similarly, sensor failures are often modeled through input noise or masking, simulating degradation in sensory or joint feedback. Policies that can maintain balance and forward motion despite these disruptions are essential for long-term autonomy in remote or mission-critical applications.

Prior work has investigated reactive strategies for fault recovery, including online adaptation, policy switching, or damage detection via latent variable inference (Cully et al., 2015; Gupta et al., 2022). However, these approaches often require additional computation, external supervision, or repeated trials to adapt. In contrast, this study focuses on proactive training methods that prepare the robot to be robust to failures from the outset, without runtime diagnostics or online learning.

2.5 Research Question

This project investigates the following core research question:

> *Can a quadruped locomotion policy trained using proactive reinforcement learning strategies—specifically curriculum-based domain randomization, smooth regularization and Proximal Policy Optimization - achieve robustness to actuator failures and sensor noise, and how does its performance compare to policies trained with only domain randomization, only smooth regularization, only PPO, or neither?*

This question arises from the practical need for deployable fault-tolerant control in resource-constrained robots operating in uncertain environments. Rather than relying on online adaptation or reactive mechanisms, the focus is on training a policy offline that is robust to internal degradation through simulated fault exposure.

2.6 Hypothesis

We hypothesize that:

> **domain randomization, smooth regularization and SR^{2}L contribute independently to improved robustness, and that their combination yields the most fault-tolerant policy overall. However, ablation experiments will assess whether one component contributes more significantly than the other across different failure conditions.**

This hypothesis will be tested through controlled simulation experiments that compare the two training strategies across a variety of fault scenarios. Metrics such as cumulative reward, locomotion success, and failure frequency will be used to evaluate performance under both clean and degraded conditions.

2.7 Research Gap

While Domain Randomization is widely used to enhance sim-to-real transfer, it is usually limited to modeling minor environmental variability — such as changes in mass or friction — rather than severe failure cases like actuator faults or sensor loss. Recent work by Liu et al. (2023a) has demonstrated the potential of damage-aware training for actuator faults, but such approaches often neglect the simultaneous presence of sensor degradation, which is a realistic consideration.

Similarly, while techniques like Smooth Regularized Reinforcement Learning (SR^2L) offer improved policy smoothness under observation noise, they have largely been evaluated in isolation, without integration into broader robustness frameworks like Domain Randomization. To date, there has been limited work examining the combined use of DR and SR^2L for fault-tolerant locomotion — particularly in lightweight robotic platforms such as the Ant robot, where computational constraints preclude reactive or adaptive control strategies at runtime.

This project addresses this gap by developing and evaluating a proactive reinforcement learning pipeline that integrates Domain Randomization for actuator faults and SR^2L for sensor noise robustness. by comparing this approach against reduced variants lacking domain randomization, smooth regularization, or both. The study aims to provide empirical evidence for the value of unified, pre-trained robustness strategies, while also producing a deployable locomotion policy suitable for low-resource quadruped systems.

As shown in Table 2.1, domain randomization and PPO have been widely used for robust locomotion. However, very few works combine DR with smooth policy regularization (SR^2L), particularly in the context of internal hardware faults. This project fills that gap by applying both techniques to a compact quadruped robot under actuator dropout and sensor noise.

Table 2.1: Summary of DRL strategies in fault-tolerant locomotion literature

Paper	**RL Algorithm**	**Robustness Strategy**	**Target Platform**
Saving the Limping (Liu et al., 2023)	PPO	Domain Randomization	Simulated quadruped
SR²L (Liu et al., 2023)	PPO, SAC	Smooth regularization (SR²L)	Mujoco-based walker/hopper
Robust Locomotion in the Wild (Peng et al., 2020)	PPO	DR + terrain randomization + perturbation curriculum	ANYmal (real robot)
Learning Agile Locomotion (Hwangbo et al., 2019)	PPO	Sensor noise + randomization	Quadruped (HyQ)
Our Proposed Work	PPO	Domain Randomization + SR²L	RealAnt (MuJoCo + real)

Chapter 3

Methodology

3.1 Simulation Setup

The choice of Proximal Policy Optimization (PPO) as the base algorithm is motivated by its strong empirical performance and sample efficiency in high-dimensional continuous control tasks (Schulman et al., 2017), especially in MuJoCo-based locomotion. Domain Randomization (DR) is employed as a proactive robustness tool, exposing the agent to actuator and sensor faults during training and enabling generalization to real-world uncertainty. Smooth Regularized Reinforcement Learning (SR^2L) complements this by stabilizing policy outputs in response to noisy inputs, enforcing smoothness through Lipschitz continuity. The combination of PPO, DR, and SR^2L forms a lightweight yet effective training pipeline for developing robust locomotion policies without relying on reactive online adaptation.

All experiments will be conducted in simulation using the MuJoCo physics engine, which supports high-fidelity contact dynamics and has been widely adopted in reinforcement learning research for robotics (Todorov et al., 2012). The simulated robot used in this project is the RealAnt model, a compact four-legged platform developed by Ote Robotics and available through the open-source RealAnt-RL framework (Robotics and Lab, 2022). The simulation mirrors the physical robot available at the University of the Witwatersrand, with matching dimensions and joint configurations.

The RealAnt robot has eight degrees of freedom, controlled via eight actuated joints (two per leg), and features a lightweight body designed for learning-based locomotion. The MuJoCo-based simulation includes contact-aware dynamics, torque control, and ArUco-based vision tracking in the physical system, though this project will focus on proprioceptive inputs only.

To simulate real-world degradation, the environment will be extended to support:

- **Actuator faults**: modeled as random joint locking events in which selected motors are disabled and fixed in place for the remainder of the episode.
- **Sensor noise**: implemented as additive Gaussian noise injected into joint position, velocity, and orientation measurements to simulate degraded proprioception.

The default terrain will be a flat surface to isolate internal fault effects. A goal velocity between 0.5–1.0 m/s will be maintained during training and evaluation. Policies will be assessed over fixed-length episodes based on performance metrics detailed in Chapter 4.

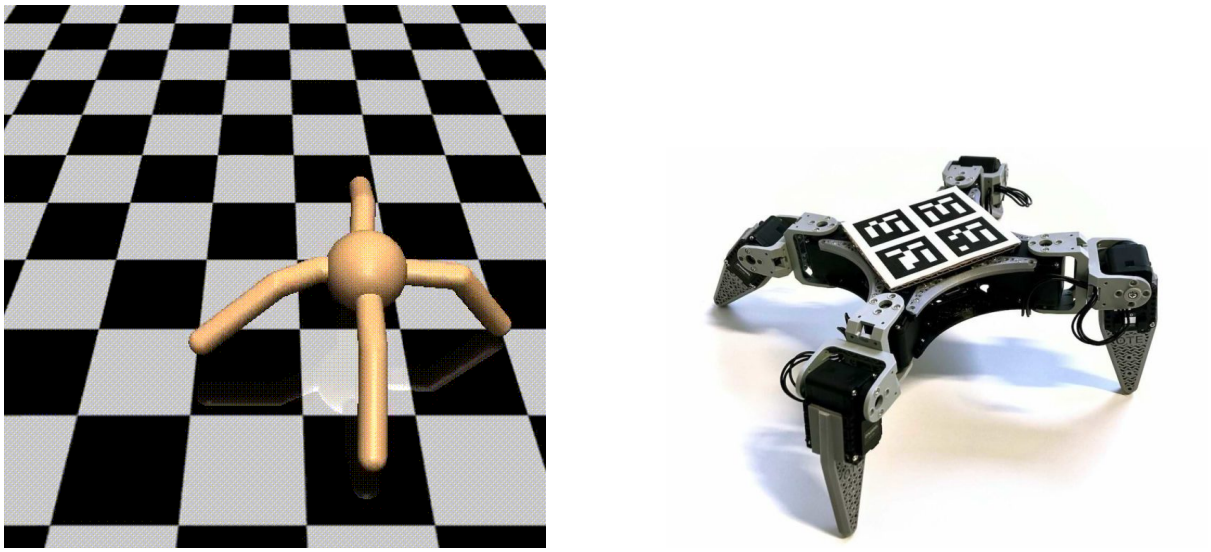

Figure 3.1: Left: RealAnt MuJoCo simulation; Right: Corresponding physical robot from Ote Robotics.

3.2 Policy Architecture and Algorithms

The control policy in this project is trained using Proximal Policy Optimization (PPO), a widely adopted actor-critic algorithm that balances exploration and stability in reinforcement learning (Schulman et al., 2017). PPO builds on the policy gradient framework by using a clipped objective function to restrict policy updates within a small trust region. This avoids destabilizing the learning process, which is especially critical in robotics, where small policy changes can lead to large variations in physical behavior.

At each iteration, PPO collects trajectories using the current policy π_θ, computes an

advantage estimate $\hat{A}_t$ based on the Generalized Advantage Estimation (GAE) method, and optimizes the following clipped objective:

$$\mathcal{L}_{\text{PPO}} = E_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right], \tag{3.1}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio between new and old policies, and ϵ is a clipping threshold typically set to 0.1 or 0.2.

The policy network architecture is a multilayer perceptron (MLP) with two hidden layers of 64 and 128 ReLU units. It maps a 28-dimensional observation vector (including joint angles, joint velocities, base orientation, and contact sensors) to an 8-dimensional continuous action vector representing joint torques for the RealAnt robot. The critic network shares the same feature encoder but predicts scalar value estimates used in advantage computation.

To further improve robustness to sensor noise and reduce high-frequency oscillations in the action space, we integrate Smooth Regularized Reinforcement Learning (SR^2L) (Liu et al., 2023c). SR^2L introduces an additional term that penalizes the variance of the policy output under small perturbations of the input state:

$$\mathcal{L}_{\text{smooth}} = E_{s\sim\mathcal{D}} \left[\|\pi_\theta(s) - \pi_\theta(s+\delta)\|^2 \right], \tag{3.2}$$

where δ is a small, zero-mean Gaussian noise vector sampled independently per batch. This encourages the policy to produce similar actions for neighboring states, enforcing local Lipschitz continuity and mitigating erratic responses due to noisy observations.

The final loss combines both objectives:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{PPO}} + \lambda \cdot \mathcal{L}_{\text{smooth}}, \tag{3.3}$$

where λ is a tunable hyperparameter that controls the trade-off between reward maximization and output smoothness. Integrating SR^2L into the PPO framework enhances training stability and leads to more robust policies under sensor uncertainty — useful in compute-limited platforms like RealAnt where online adaptation is not an option.

3.3 Fault Injection: Limb Dropout and Sensor Noise

To evaluate and improve the robustness of the learned policy, the simulation environment is extended to support two forms of internal faults commonly encountered in real-world robotic deployments: actuator (limb) dropout and sensor noise.

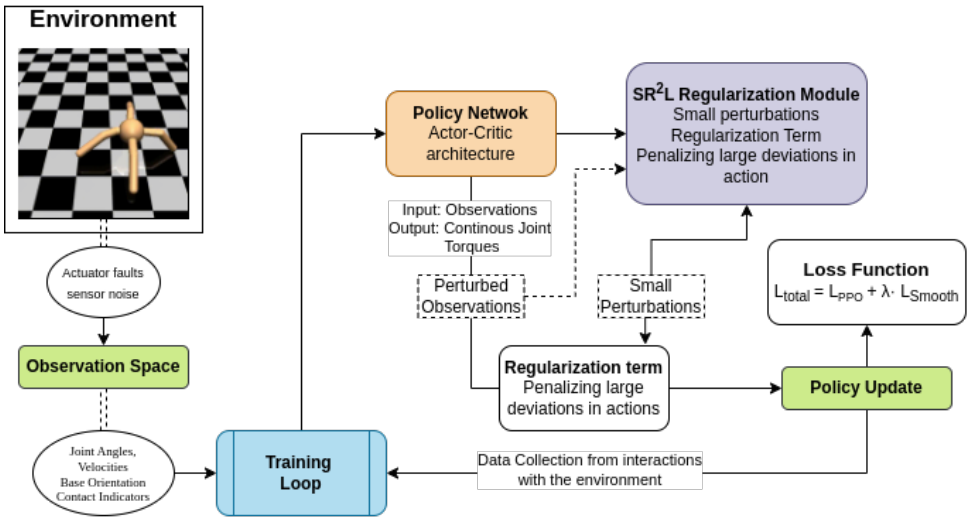

Figure 3.2: Policy training pipeline combining PPO with SR^{2}L smooth regularization under actuator and sensor perturbations.

Actuator Faults (Limb Dropout)

Actuator faults are modeled by disabling one or more joints at random during each training episode. This is implemented by setting the corresponding joint torque to zero and freezing its position for the remainder of the episode. Affected joints become kinematically locked, simulating mechanical failure or power loss. The number and identity of affected joints are randomized per episode, encouraging the policy to generalize to a range of possible limb dropout configurations. This randomization is needed to avoid overfitting the policy to a single failure pattern (Liu et al., 2023a).

Sensor Noise

Sensor degradation is modeled by injecting zero-mean Gaussian noise into the robot's sensory inputs, including joint angles and joint velocities. At each timestep, the observation s_t becomes:

$$\tilde{s}_t = s_t + \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, \sigma^2 I), \tag{3.4}$$

where σ is tuned based on realistic sensor error characteristics. This approach captures degraded or unstable sensor feedback caused by hardware limitations, wear, or environmental interference. During training, both σ and the dropout probability are gradually increased to expose the policy to higher-fidelity and lower-fidelity signals alike.

These fault injection mechanisms are integrated into the simulation as part of the domain randomization process. By explicitly incorporating failures at the input and actuation levels, the learning process becomes more resilient and yields policies that can gracefully

degrade in the presence of internal faults.

3.4 Training Procedures

The control policy is trained using standard PPO rollouts over episodic interactions. To enable robust adaptation, we incorporate actuator faults and sensor noise during training, as discussed in Section 3.3. However, instead of applying these perturbations randomly from the beginning, we adopt a curriculum-based domain randomization (CDR) strategy that gradually increases complexity over time.

Why Curriculum Learning?

A novel approach to domain randomization would inject multiple, randomly selected faults into each training episode from the outset. While this ensures broad coverage, it can significantly increase task variance and lead to unstable or inefficient learning — especially in environments with sparse or delayed rewards (Bengio et al., 2009; Peng et al., 2020). Policies may struggle to discover stable locomotion patterns when they must simultaneously account for many diverse failure modes, leading to convergence on overly conservative or suboptimal behaviors.

Instead, curriculum learning begins with simpler fault conditions and incrementally introduces complexity. This allows the policy to:

- First acquire basic locomotion behavior in a nearly ideal setting.
- Learn compensatory strategies for single faults in isolation.
- Gradually develop fault-tolerant gait adjustments under increasingly severe degradation.

This staged learning approach reduces the initial difficulty of the task and improves sample efficiency, particularly in early training. It is widely supported by prior work on robust control and transfer learning in robotics (Peng et al., 2018; Jain et al., 2020; Miki et al., 2022).

Curriculum vs. Naive Randomization: Comparative Summary

Training Schedule

Table 3.1: Comparison between Curriculum-Based vs. Naive Domain Randomization for Robust Locomotion

Criterion	**Curriculum-Based DR**	**Novel Randomization**
Initial difficulty	Low (starts simple)	High (complex failures from start)
Sample efficiency	High during early training	Low; agent may fail to explore usefully
Policy stability	Stable incremental learning	Prone to instability / reward collapse
Generalization capacity	High, due to gradual exposure	Moderate; may learn conservative or averaged strategies
Implementation effort	Moderate (requires tuning)	Simple (just inject random faults)

Each training episode consists of 500 timesteps. The training curriculum progresses as follows:

- **Phase 1: Warm-up (Epochs 0–200)** - No actuator faults, minimal sensor noise - Goal: Learn base locomotion on flat terrain
- **Phase 2: Isolated Faults (Epochs 200–600)** - Single fixed joint dropout per episode (varies between episodes) - Low to moderate sensor noise - Goal: Learn compensatory behavior per failure type
- **Phase 3: Full Randomization (Epochs 600+)** - Multiple joint faults and randomized dropout probability - High sensor noise variance - Goal: Maximize robustness across unpredictable failures

This curriculum strategy helps stabilize early training and prepares the agent to recover from more severe disruptions in later phases.

As illustrated in Figure 3.3, the agent is exposed to increasingly complex scenarios in a staged manner. Each phase is designed to build stability and fault tolerance before progressing to more challenging conditions.

SR^{2}L Integration

During each PPO update step, a batch of perturbed observations is used to compute the SR^{2}L loss term. This regularization is applied throughout training, including early phases, to ensure smooth action transitions even when sensory input is clean. The weight

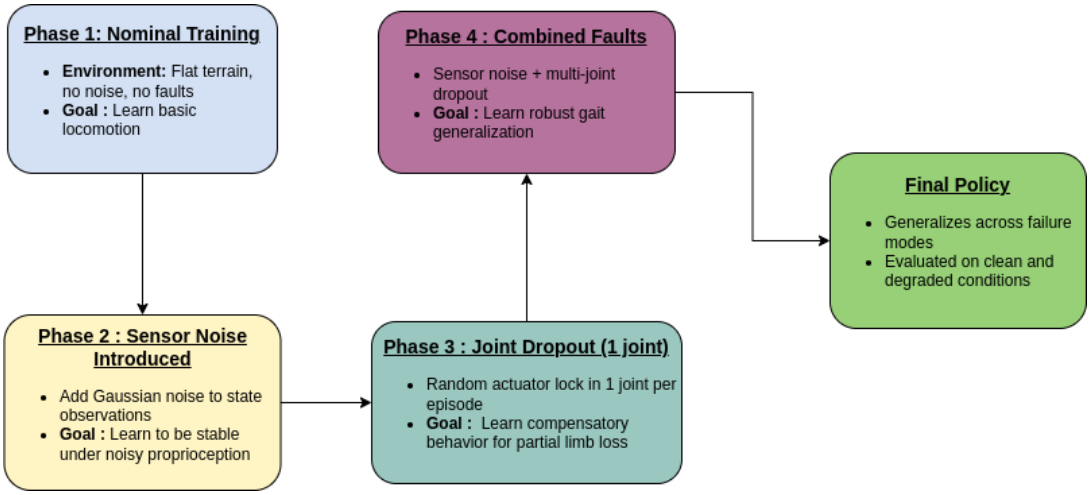

Figure 3.3: Curriculum-based training pipeline for robust locomotion. Each phase increases difficulty by progressively adding noise and actuator faults. The agent learns to stabilize under each condition before progressing.

$\lambda = 0.01$ is selected based on prior studies and tuned empirically to balance performance and smoothness (Liu et al., 2023c).

Hyperparameters

Key training hyperparameters follow commonly used settings in PPO literature and are consistent with configurations reported in Hwangbo et al. (2019):

- PPO learning rate: 3×10^{-4}
- Batch size: 2048
- Epochs per update: 10
- Clipping parameter (ϵ): 0.2
- Discount factor (γ): 0.99
- GAE parameter: 0.95
- SR^2L regularization coefficient (λ): 0.01

All training is implemented in PyTorch using the ‘stable-baselines3‘ PPO interface, customized for MuJoCo RealAnt integration and curriculum-based randomization.

3.5 Ablation Baselines

To evaluate the contribution of each component in the proposed robustness pipeline, we compare the full policy—trained with both curriculum-based domain randomization (DR) and smooth regularization (SR^2L)—against three ablated variants:

- **PPO only**: A baseline policy trained without domain randomization or smooth regularization (i.e., $\lambda = 0$), under ideal, fault-free conditions. This reflects the standard RL training setup (Schulman et al., 2017).
- **PPO + DR**: A policy trained using domain randomization (actuator dropout and sensor noise), but without SR^2L. This isolates the effect of curriculum-based failure exposure.
- **PPO + SR^2L**: A policy trained with smooth regularization enabled but no fault injection or domain randomization. This evaluates the impact of regularizing policy outputs in the absence of explicit fault training.

All policies share the same neural architecture, training schedule, and hyperparameters to ensure a controlled comparison. The only difference lies in the application of DR and SR^2L during training.

Evaluation Scenarios

Each policy is evaluated under a consistent set of simulated fault scenarios:

- **Single joint dropout**: A single actuator is randomly locked for each episode.
- **Multiple joint dropout**: Two or more joints are disabled, simulating compounded damage.
- **Sensor noise injection**: Zero-mean Gaussian noise added to joint readings and base orientation, with increasing variance.
- **Clean evaluation**: No noise or faults, to assess if robustness affects performance in ideal conditions.

Each condition is tested over 100 rollouts using consistent random seeds for reproducibility. Results are averaged and compared using performance metrics detailed in Chapter 4, including reward, task success rate, gait stability, and recovery time.

3.6 Stretch Goals and Extended Objectives

While the primary focus of this research is to train a locomotion policy robust to joint dropout and sensor noise using Domain Randomization and SR^{2}L, we identify several promising extensions that could be pursued if time and computational resources permit.

1. Terrain-Aware Policy Adaptation

In realistic deployment scenarios, robots often face non-flat terrains, such as slopes, stairs, or unstable surfaces. As an extension, we aim to modify the simulation environment to include multiple paths to a goal — such as a short staircase vs. a longer slope — and train the policy to choose paths adaptively based on its internal damage condition. This would require encoding terrain features as additional state inputs or extending the observation space with terrain elevation maps.

2. Vision-Based Terrain Estimation

A longer-term goal is to eliminate reliance on hardcoded terrain labels by integrating onboard visual input (e.g., simulated RGB or depth camera) into the observation space. Using vision, the robot could autonomously perceive terrain complexity and select traversal strategies accordingly. This aligns with real-world mission-critical scenarios where full environment models may not be known a priori.

3. Sim-to-Real Transfer Validation

Finally, we aim to export the trained policy to the physical RealAnt platform and conduct limited real-world tests under safe actuator and sensor degradation scenarios.

Chapter 4

Evaluation

This chapter outlines the evaluation framework used to assess the performance of the proposed robust locomotion policy and its ablated variants. The goal is to quantify the impact of actuator and sensor faults on policy effectiveness, and to determine the contribution of each component—domain randomization and smooth regularization—to overall robustness.

To achieve this, we compare four policies:

- PPO only (no robustness strategies)
- PPO + Domain Randomization
- PPO + Smooth Regularization (SR^2L)
- PPO + Domain Randomization + SR^2L (full policy)

Each policy is evaluated under identical fault conditions and measured using quantitative metrics such as reward, recovery time, success rate, and gait stability. This comparative analysis enables a deeper understanding of the individual and combined benefits of proactive robustness techniques.

4.1 Evaluation Metrics

To comprehensively evaluate policy performance, we use the following metrics:

- **Success Rate**: Fraction of test episodes in which the robot maintained forward locomotion above a predefined threshold (e.g., 1.5 meters within 5 seconds).
- **Cumulative Reward**: Sum of per-step rewards accumulated during each episode. Shows the efficiency and stability.
- **Recovery Time**: Time required to resume forward locomotion after a fault is introduced.

- **Failure Rate**: Percentage of episodes where the robot collapsed, stopped moving, or spun in place for more than 2 seconds.

4.2 Experimental Setup

We compare the full policy (PPO + DR + SR^{2}L) against three reduced variants:

- PPO + Domain Randomization only
- PPO + SR^{2}L only
- PPO without any robustness strategies (clean baseline)

This ablation allows us to isolate the impact of each component on final performance.

Test Scenarios

- Clean environment (no faults)
- Single joint locked (random)
- Multiple joint lock (2–3 joints)
- Sensor noise only
- Combined faults (joint lock + noise)

Policies are evaluated in MuJoCo using the same simulation settings as during training, but with fault configurations held constant for all test agents in each trial batch.

4.3 Result Analysis Strategy

All metrics are averaged over 100 test rollouts per condition. For numerical metrics (e.g., cumulative reward, gait stability), we compute:

- Mean and standard deviation across runs
- 95% confidence intervals
- Paired t-tests between robust and baseline agents to assess statistical significance

For categorical metrics (e.g., success/failure), chi-squared tests are used to assess significant differences in proportions.

Results will be visualized using:

- Bar plots for success rates and failure rates

- Box plots for cumulative rewards and gait stability
- Line graphs showing performance degradation as noise or fault severity increases

4.4 Interpretation and Limitations

Performance differences will be interpreted in terms of generalization, fault tolerance, and policy smoothness. Any robustness gains should not significantly reduce clean-environment performance, to ensure no unnecessary trade-offs are introduced. Limitations include:

- Evaluation is restricted to simulation; real-world performance may differ.
- Gait quality and energy efficiency are not explicitly optimized.
- Robustness is evaluated on predefined failure modes; unseen faults may behave differently.

4.5 Timeline and Milestones

The following timeline outlines the major milestones for this project, from initial implementation to final submission. The structure allows for iterative development while ensuring that evaluation and documentation are completed well before the deadline.

Table 4.1: Updated Project Schedule (27 June – 14 October 2025)

Time Period	**Milestone**
June 27 – July 5	Finalize methodology, implementation plan, and test PPO agent for baseline performance.
July 6 – July 15	Implement actuator dropout and sensor noise; begin curriculum-based domain randomization (DR).
July 16 – July 25	Integrate SR^{2}L into PPO. Tune hyperparameters and ensure training stability.
July 26 – August 5	Train final robust agent (PPO + DR + SR^{2}L) under full curriculum.
August 6 – August 15	Train and evaluate ablated agents: PPO only, PPO + DR, and PPO + SR^{2}L.
August 16 – August 25	Perform structured evaluations under all test conditions. Log results, compute metrics.
August 26 – September 5	Analyze results. Generate tables, charts, and statistical comparisons.
September 6 – September 15	Draft Chapters 4–5 (Evaluation and Discussion). Review Methodology.
September 16 – September 25	Write Introduction, Background, and Related Work. Insert figures, finalize formatting.
September 26 – October 5	Integrate feedback, revise writing, attempt stretch goal (e.g., terrain-aware or vision-based variant).
October 6 – October 14	Final proofreading, complete references, and submit final research report.

Chapter 5

Limitations

While this project aims to develop a fault-tolerant locomotion policy using proactive reinforcement learning strategies, several practical and external limitations must be acknowledged:

- **Fixed fault model scope:** Only actuator locking and Gaussian sensor noise are modeled. Other real-world degradations—such as mechanical wear, control latency, or sensor drift over time—are outside the scope of the current simulation platform.
- **Hardware-dependent constraints:** The policy is designed to fit on-board compute resources, but actual deployment on the physical robot is untested due to limited access to deployment tooling, hardware debuggers, and power-safe integration.
- **Limited failure coverage:** While curriculum-based training introduces various fault configurations, the agent is not exposed to all possible fault permutations or emergent compound failures that might occur in unconstrained environments.
- **No formal robustness guarantees:** The approach provides empirical robustness through training exposure but does not guarantee bounded error or fail-safe behavior under safety-critical constraints.

Despite these limitations, the project establishes a foundation for future real-world evaluation, terrain-aware decision making, and closed-loop fault detection systems on resource-constrained quadruped robots.

Chapter 6

Conclusion

This research project aims to address the critical challenge of actuator and sensor fault tolerance in quadruped robots by leveraging proactive reinforcement learning techniques. Using the RealAnt platform as a case study, we propose a policy training pipeline that combines Proximal Policy Optimization, curriculum-based domain randomization with smooth regularization (SR^2L) to produce robust, adaptable locomotion controllers.

While prior work has demonstrated the efficacy of domain randomization and smoothness regularization independently, this study is among the first to systematically integrate them for internal fault robustness in small, resource-constrained legged robots. The resulting policy is expected to outperform standard PPO baselines under joint failure and sensor noise, without sacrificing clean-environment performance — a key requirement for mission-critical deployment.

The research design also includes meaningful extensions such as terrain-aware policy adaptation and sim-to-real transfer, aligning this work with the broader goal of long-term robotic autonomy in unstructured environments. Through rigorous evaluation, statistical analysis, and comparative benchmarking.

Bibliography

Yoshua Bengio, Jerome Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. *Proceedings of the 26th annual international conference on machine learning*, pages 41–48, 2009.

Antoine Cully, Jeff Clune, Danesh Tarapore, and Jean-Baptiste Mouret. Robots that can adapt like animals. *Nature*, 521(7553):503–507, 2015.

Avinash Gupta, Kunal Alwala, Adam Margolis, Erwin Coumans, Jie Tan, and Xue Bin Peng. Meta-rl sim2real for vision-based legged locomotion. In *Robotics: Science and Systems (RSS)*, 2022.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, pages 1861–1870. PMLR, 2018.

Jemin Hwangbo, Joonho Lee, Alexey Dosovitskiy, Dario Bellicoso, Vassilios Tsounis, Vladlen Koltun, and Marco Hutter. Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26):eaau5872, 2019.

Aayush Jain, Adam Margolis, Xue Bin Peng, Jie Tan, and Sergey Levine. Learning and recovering locomotion on degraded robots. In *Conference on Robot Learning (CoRL)*, pages 1316–1326, 2020.

Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning quadrupedal locomotion over challenging terrain. *Science Robotics*, 5(47): eabc5986, 2020.

Qiyang Liu, Xue Bin Peng, Sergey Levine, and Jie Tan. Saving the limping: Learning fault-tolerant control policies for quadrupedal robots using damage-aware training. *Science Robotics*, 8(74):eadd5002, 2023a.

Shuang Liu, Ziyu Li, Jing Peng, and Yisong Wen. Robust and smooth reinforcement learning via smooth regularization. *Mathematics*, 11(23):4744, 2023b.

Shuang Liu, Ziyu Li, Jing Peng, and Yisong Wen. Robust and smooth reinforcement learning via smooth regularization. *Mathematics*, 11(23):4744, 2023c.

Adam Margolis, Karen Yang, Vikash Kumar, Xue Bin Peng, Jie Tan, and Sergey Levine. Rapid motor adaptation for legged robots. In *Conference on Robot Learning (CoRL)*, pages 1247–1258, 2022.

Takahiro Miki, Joonho Lee, Jemin Hwangbo, and Marco Hutter. Learning robust perceptive locomotion for quadrupedal robots in the wild. *Science Robotics*, 7(62):eabk2822, 2022.

Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 3803–3810. IEEE, 2018.

Xue Bin Peng, Erwin Coumans, Tianhao Zhang, Tsang-Wei Edward Lee, Jie Tan, and Sergey Levine. Learning robust perceptive locomotion for quadrupedal robots in the wild. In *Robotics: Science and Systems (RSS)*, 2020.

Aravind Rajeswaran, Kendall Lowrey, Emanuel Todorov, Igor Mordatch, and Sergey Levine. Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. *arXiv preprint arXiv:1709.10087*, 2017.

Ote Robotics and AaltoVision Lab. Realant-rl: Reinforcement learning benchmarks for real-world quadrupeds. https://github.com/AaltoVision/realant-rl, 2022. Accessed May 2025.

Fereshteh Sadeghi and Sergey Levine. Cad2rl: Real single-image flight without a single real image. In *Robotics: Science and Systems (RSS)*, 2016.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. In *arXiv preprint arXiv:1707.06347*, 2017.

Richard S Sutton and Andrew G Barto. *Reinforcement Learning: An Introduction*. MIT press, 2018.

Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 23–30. IEEE, 2017.

Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012.

Wits University Faculty of Science post-graduate student AI declaration

I understand that the use of generative AI tools (such as ChatGPT or similar) without explicitly declaring such use constitutes a form of plagiarism and is classified by Wits University as academic misconduct.

I declare that in the course of conducting the research towards my degree or in the preparation of this thesis/dissertation/research report (select one by marking with an X):

I **did not** make use of generative AI tools ☐

I **did** make use of generative AI tools for the following (tick all that apply):

1. Idea Generation (research problem/design, hypothesis) ☐
2. Sourcing Related Work (summarising, identifying sources) ☐
3. Methods and Experiment Design (experiment setup, model tuning ☐
4. Data Analysis (presentation, coding, interpretation) ☐
5. Theoretical Development (theorem proving, conceptual analysis) ☐
6. Code Development (generating algorithms, writing scripts) ☐
7. Presentation (rendering graphics, formatting) ☑
8. Editing (grammar, readability) ☑
9. Writing (text generation, document structuring) ☑
10. Citation Formatting (structuring, organising) ☐

If other uses were involved, please specify below:

Generative AI tool used (list all)	Used for?

If generative AI tools were used as an integral part of the experimental design or in the direct execution of my research, I confirm that details of this use are clearly outlined in the relevant experimental/methodology chapters of my thesis/dissertation/research report.

Student number: 2561034

Candidate signature: 

Date: 26/05/2025